Machine Learning

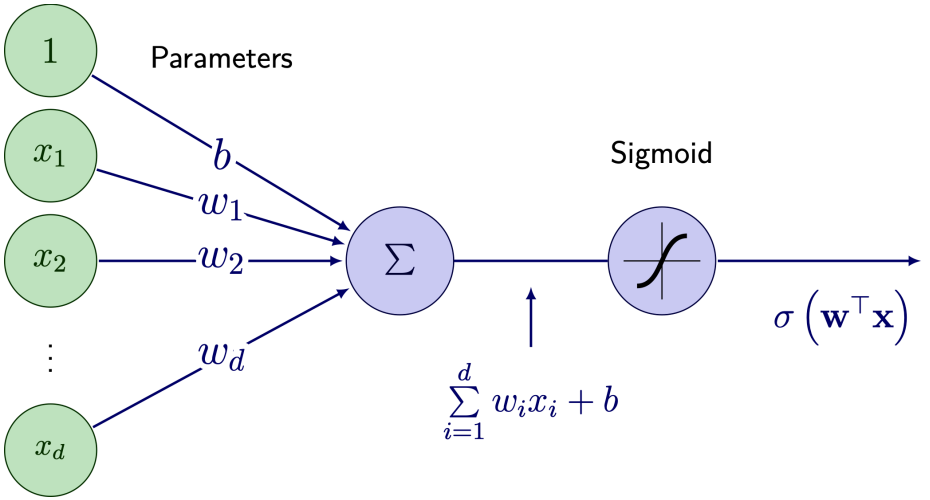
Lecture 3: Deep neural networks

Simon BERNARD
simon.bernard@univ-rouen.fr



From logistic regression to neural networks

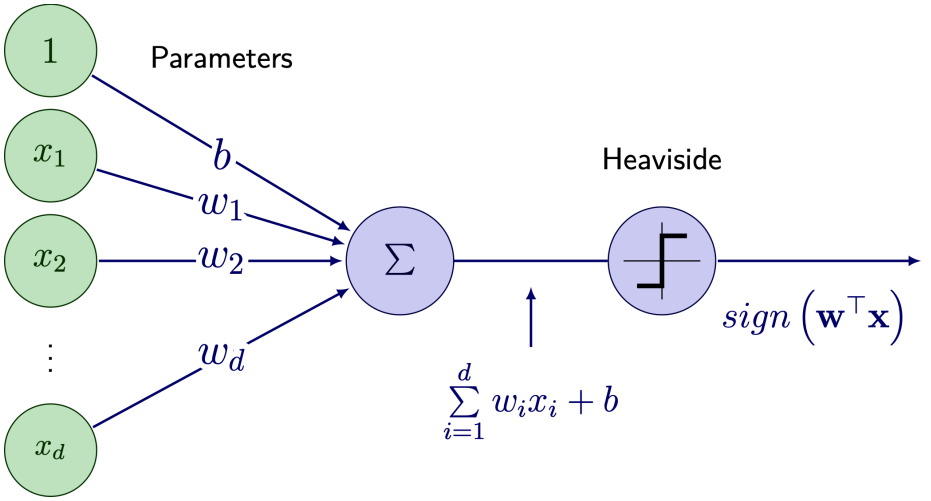
Logistic regression



- Predict $\sigma(\mathbf{w}^\top \mathbf{x}) = p(y = 1|\mathbf{x}) \in [0, 1]$
- Decision function is:

$$\hat{y} = \begin{cases} 1 & \text{if } \mathbf{w}^\top \mathbf{x} > 0 \\ 0 & \text{otherwise} \end{cases}$$

Perceptron (a.k.a. neuron)



- Predict $y \in \{0, 1\}$ instead of $p(y = 1|\mathbf{x}) \in [0, 1]$
- Decision function is:

$$\hat{y} = \begin{cases} 1 & \text{if } \mathbf{w}^\top \mathbf{x} > 0 \\ 0 & \text{otherwise} \end{cases}$$

Perceptron learning algorithm

1. Initialize $\mathbf{w}$ either with zeros or small random values
2. For each sample $(\mathbf{x}_i, y_i) \in \mathcal{D}$:
   - Compute the prediction with the current weights:

$$h(\mathbf{x}_i) = \begin{cases} 1 & \text{if } \mathbf{x}_i^\top \mathbf{w} > 0 \\ 0 & \text{otherwise} \end{cases}$$

   - Update the weights:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y_i - h(\mathbf{x}_i))\mathbf{x}_i$$

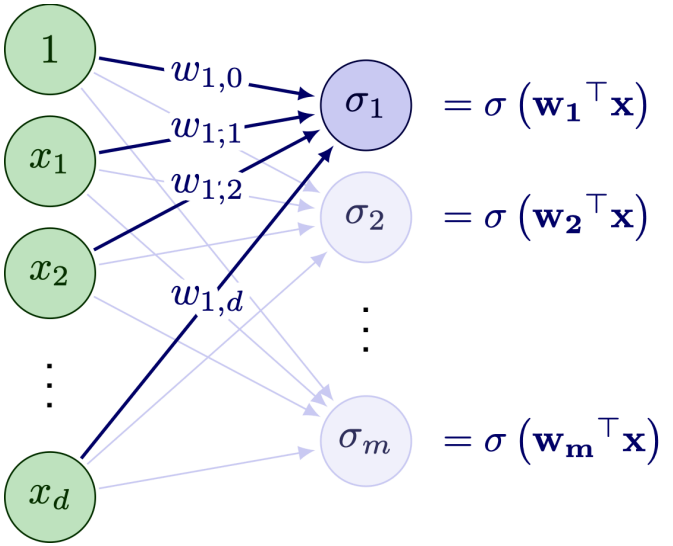
     where η is the learning rate (as in gradient descent)
3. Repeat until convergence, i.e. for a user-specified small value ϵ:

$$\frac{1}{n}\sum_{i=1}^{n} |y_i - h(\mathbf{x}_i)| < \epsilon$$

Notebook lec3-perceptron-learning.ipynb

Multi-output perceptron

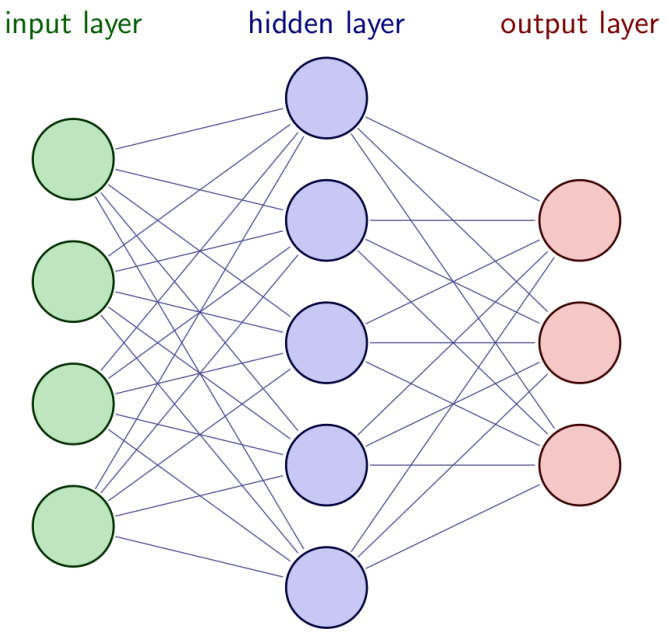
Several perceptrons can be used in parallel to predict multiple outputs (e.g. for multi-class classification)



- σ must be a nonlinear activation function
- If σ is the softmax function, we retrieve the softmax regression model

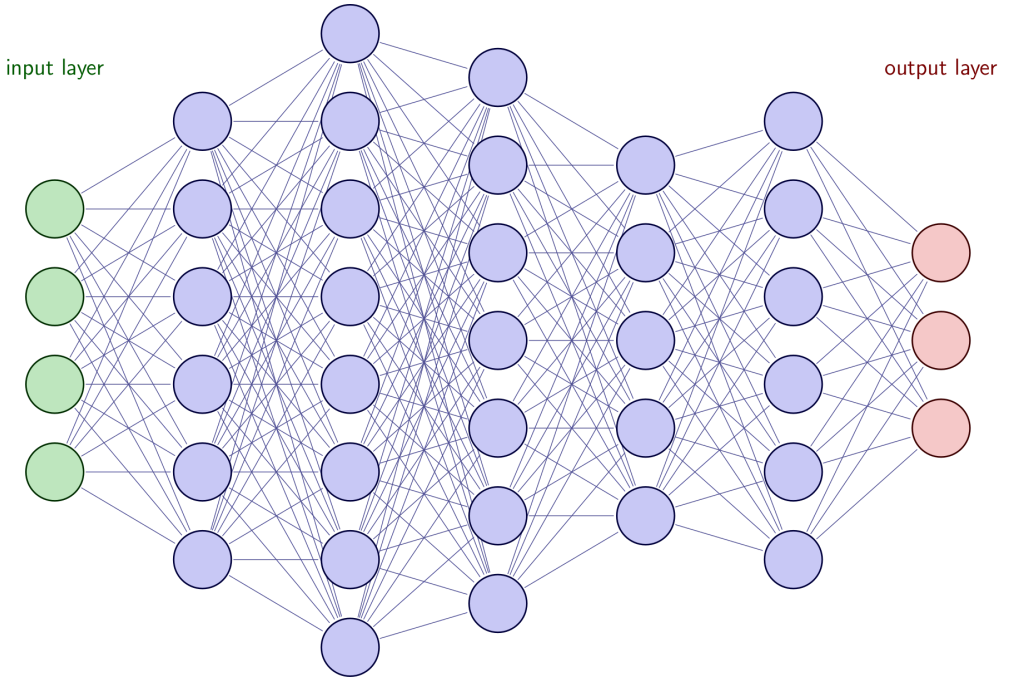
Multilayer perceptron (MLP)

MLP is a generalization where the output of each unit is an input of the next layer



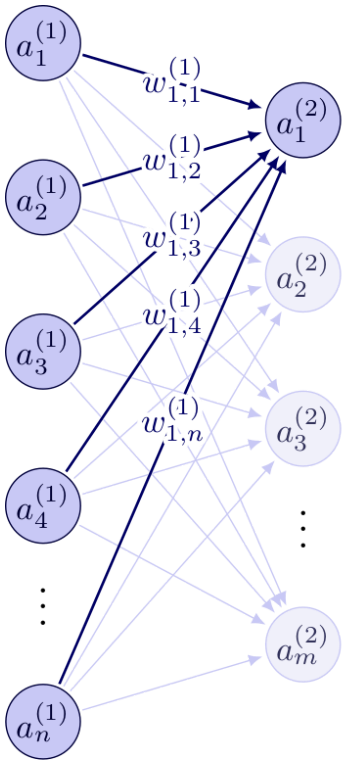
Deep Neural Network (DNN)

A Deep NN is a NN with many layers, and thus many parameters (one per connection)



Deep Neural Network (DNN)

- A layer is composed of m units (or neurons) in parallel, each with its own parameters
- The output of a layer can be computed at once using matrix multiplication and vectorization:



$$= \sigma\left(\sum_{i=1}^{n} w_{1,i}^{(1)} a_i^{(1)}\right)$$

$$\begin{pmatrix} a_1^{(2)} \\ a_2^{(2)} \\ \vdots \\ a_m^{(2)} \end{pmatrix} = \sigma\left[\begin{pmatrix} w_{1,1}^{(1)} & w_{1,2}^{(1)} & \dots & w_{1,n}^{(1)} \\ w_{2,1}^{(1)} & w_{2,2}^{(1)} & \dots & w_{2,n}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,1}^{(1)} & w_{m,2}^{(1)} & \dots & w_{m,n}^{(1)} \end{pmatrix} \begin{pmatrix} a_1^{(1)} \\ a_2^{(1)} \\ \vdots \\ a_n^{(1)} \end{pmatrix}\right]$$

$$\mathbf{a}^{(2)} = \sigma\left(\mathbf{W}^{(1)}\mathbf{a}^{(1)}\right)$$

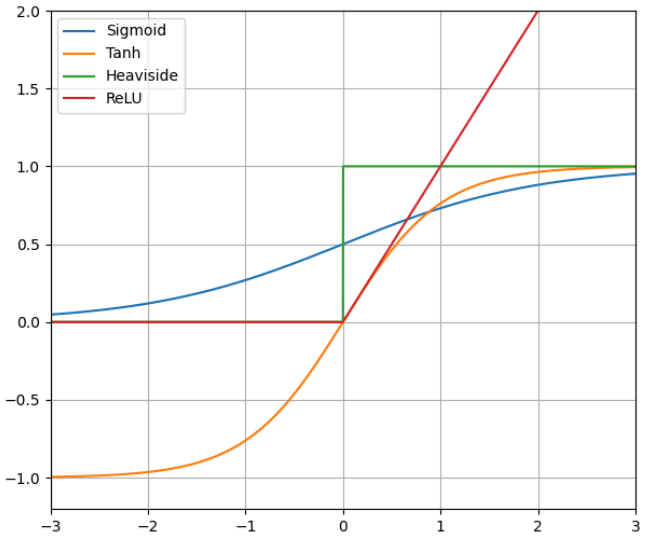
Deep Neural Network (DNN)

- A DNN is composed of L layers in sequence, such that:

$$
\begin{aligned}
\mathbf{a}^{(0)} &= \mathbf{x} \\
\mathbf{a}^{(1)} &= \sigma^{(1)}(\mathbf{W}^{(1)}\mathbf{a}^{(0)}) \\
\mathbf{a}^{(2)} &= \sigma^{(2)}(\mathbf{W}^{(2)}\mathbf{a}^{(1)}) \\
&\vdots \\
\mathbf{a}^{(L)} &= \sigma^{(L)}(\mathbf{W}^{(L)}\mathbf{a}^{(L-1)}) \\
h(\mathbf{x}) = \hat{\mathbf{y}} &= \mathbf{a}^{(L)}
\end{aligned}
$$

- $\mathbf{W}^{(l)}$ is the weight matrix of layer l, to be learnt
- $\sigma^{(l)}$ is the activation function of layer l and is not necessarily the same for all layers
- For classification tasks, $\sigma^{(L)}$ is the sigmoid or the softmax function
- For regression tasks, $\sigma^{(L)}$ is the identity function

Activation functions



- Sigmoid:

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

- Hyperbolic tangent:

$$\sigma(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

- ReLU:

$$\sigma(x) = \max(0, x)$$

- Heaviside:

$$\sigma(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Training a deep neural network

Parameter learning

- Parameters: $\mathbf{W} = \{\mathbf{W}^{(1)}, \ldots, \mathbf{W}^{(L)}\}$ (weights on the connections between units)
- We want to find the $\mathbf{W}$ that minimize a loss function:

$$LL(\mathbf{W}) = \sum_{i=1}^{n} L(y_i, h(\mathbf{x}_i))$$

- Any differentiable loss function can be used, but the cross-entropy loss is the most common for classification tasks and the mean squared error for regression tasks
- For the same reasons as with logistic regression, we use the **gradient descent** method
- To do so, we need to compute the gradient of the loss function w.r.t. the parameters

Forward pass

- For one sample $\mathbf{x}$ and a L-layers network, we have:

$$\mathbf{x} = \mathbf{a}^{(0)} \xrightarrow{\mathbf{W}^{(1)}} \mathbf{s}^{(1)} \xrightarrow{\sigma^{(1)}} \mathbf{a}^{(1)} \rightarrow \ldots \rightarrow \mathbf{a}^{(L-1)} \xrightarrow{\mathbf{W}^{(L)}} \mathbf{s}^{(L)} \xrightarrow{\sigma^{(L)}} h(\mathbf{x}) = \hat{\mathbf{y}}$$

  with, $\forall l \in \{1, \ldots, L\}$:

$$\mathbf{s}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)}$$
$$\mathbf{a}^{(l)} = \sigma^{(l)}(\mathbf{s}^{(l)})$$

  and with the output of the network being $h(\mathbf{x}) = \hat{\mathbf{y}} = \mathbf{a}^{(L)}$

- This is called the **forward pass**

Chain rule

- $L(y_i, h(\mathbf{x}_i))$ is expressed as a function of h, which depends on L and $\sigma^{(l)}$:

$$h(\mathbf{x}_i) = \sigma^{(L)}(\mathbf{W}^{(L)}\sigma^{(L-1)}(\mathbf{W}^{(L-1)} \dots \sigma^{(1)}(\mathbf{W}^{(1)}\mathbf{x}_i) \dots))$$

- **How do you calculate the gradient?**
- Solution: use the **chain rule** of differential calculus

$$\frac{\partial f}{\partial w_i} = \sum_k \frac{\partial f}{\partial a_k} \underbrace{\frac{\partial a_k}{\partial w_i}}_{recurrence}$$

where a_k are the input functions of f (e.g. units in the previous layer)

Chain rule

- Simplified example: a 2-hidden layer MLP, with $x, w_1, w_2, w_3 \in \mathbb{R}$ such that:

$$x \xrightarrow{w_1} s^{(1)} \xrightarrow{\sigma^{(1)}} a^{(1)} \xrightarrow{w_2} s^{(2)} \xrightarrow{\sigma^{(2)}} a^{(2)} \xrightarrow{w_3} s^{(3)} \xrightarrow{\sigma^{(3)}} h(x) = \hat{y}$$

We have:

$$\begin{aligned} h(x) &= \sigma^{(3)}(s^{(3)}) \\ s^{(3)} &= w_3 a^{(2)} \\ a^{(2)} &= \sigma^{(2)}(s^{(2)}) \\ s^{(2)} &= w_2 a^{(1)} \\ a^{(1)} &= \sigma^{(1)}(s^{(1)}) \\ s^{(1)} &= w_1 x \end{aligned}$$

$$\begin{aligned} \frac{\partial h(x)}{\partial w_1} &= \frac{\partial h(x)}{\partial s^{(3)}} \frac{\partial s^{(3)}}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial s^{(2)}} \frac{\partial s^{(2)}}{\partial a^{(1)}} \frac{\partial a^{(1)}}{\partial s^{(1)}} \frac{\partial s^{(1)}}{\partial w_1} \\ &= \frac{\partial \sigma^{(3)}(s^{(3)})}{\partial s^{(3)}} w_3 \frac{\partial \sigma^{(2)}(s^{(2)})}{\partial s^{(2)}} w_2 \frac{\partial \sigma^{(1)}(s^{(1)})}{\partial s^{(1)}} x \end{aligned}$$

Backpropagation

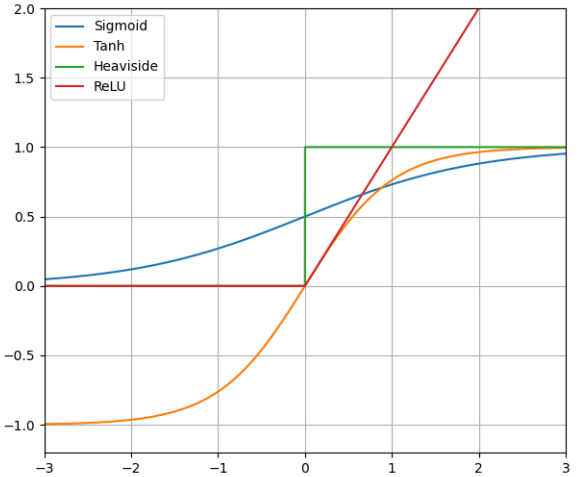
- This is called the **backward pass or backpropagation**

$$x \xleftarrow{\frac{\partial s^{(1)}}{\partial w_1}} s^{(1)} \xleftarrow{\frac{\partial a^{(1)}}{\partial s^{(1)}}} a^{(1)} \xleftarrow{\frac{\partial s^{(2)}}{\partial a^{(1)}}} s^{(2)} \xleftarrow{\frac{\partial a^{(2)}}{\partial s^{(2)}}} a^{(2)} \xleftarrow{\frac{\partial s^{(3)}}{\partial a^{(2)}}} s^{(3)} \xleftarrow{\frac{\partial h(x)}{\partial s^{(3)}}} h(x) = \hat{y}$$

- $\sigma^{(l)}$ **must be differentiable**
- The same principle applies to calculating the gradient of the loss function
- This can be generalised with several neurons and several layers
- This can also be extended to tensor calculus

Activation functions

- Heaviside: not differentiable
- Sigmoid and tanh: differentiable, but derivative shrink to zero as the number of layers increases [(1)]
- ReLU: differentiable, but zero derivatives for any values ≤ 0 [(2)]
- Variants of ReLU (e.g. LeakyReLU) solve the dying ReLU problem in modern deep NN



(1) Not suitable for modern DNN. This problem is called the *vanishing gradient* problem

(2) Can "deactivate" the neuron when large gradients are propagated. This is called the *dying ReLU* problem

Automatic differentiation

> Automatic differentiation (AD) is a set of techniques to **numerically evaluate the derivative** of a function specified by a computer program. It exploits the fact that **computer program executes a sequence of elementary arithmetic operations** (addition, subtraction, multiplication, division, etc.) and elementary functions (exp, log, sin, cos, etc.). By applying the **chain rule to these operations**, derivatives of arbitrary order can be computed automatically.

- Modern DL frameworks implement automatic differentiation
- Allows to compute the gradient of the loss function w.r.t. the parameters of the network
- Enables the use of gradient descent methods for training

DNN in practice

DNN Softwares

- Implementing and using DNN is made easy by modern frameworks













- Examples in this lecture are made with Keras, a high-level API for TensorFlow
- Low-level APIs (e.g. PyTorch, TensorFlow) are more flexible, and for more complex architectures

Example with Keras

```
from keras.models import Sequential
from keras.layers import Dense, Activation

model = Sequential()
model.add(Dense(128, input_dim=X.shape[1]))     # hidden layer with 128 units
model.add(Activation('relu'))                   # ReLU activation function
model.add(Dense(128))                           # hidden layer with 128 units
model.add(Activation('relu'))                   # ReLU activation function
model.add(Dense(10))                            # 10 output units for 10 classes
model.add(Activation('softmax'))                # softmax activation function

model.compile(optimizer='rmsprop', loss='categorical_crossentropy')

print("Training...")
model.fit(X, y, epochs=10, batch_size=16)
```

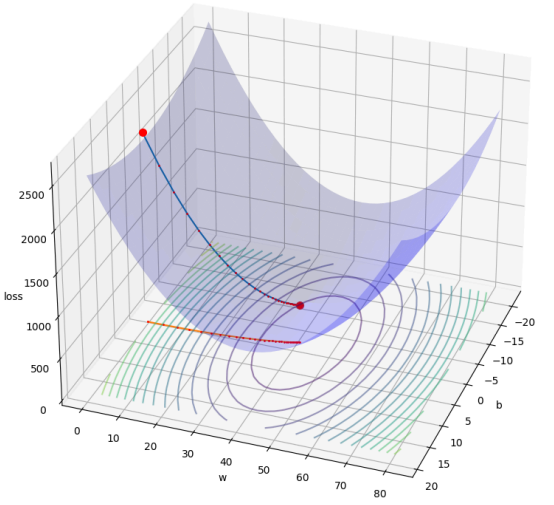
- Many choices: number of layers/units, activation functions, loss function, optimizer, etc.
- What is behind the `fit` method?

Reminder on gradient descent methods

- **Batch gradient descent**: approx. the gradient on the entire dataset

$$\nabla LL(\mathbf{W}) = \frac{1}{n}\sum_{i=1}^{n} \nabla_{\mathbf{w}} L(y_i, h_{\mathbf{W}}(\mathbf{x}_i))$$

$$\mathbf{W}_{k+1} \leftarrow \mathbf{W}_k - \eta \nabla LL(\mathbf{W}_k)$$

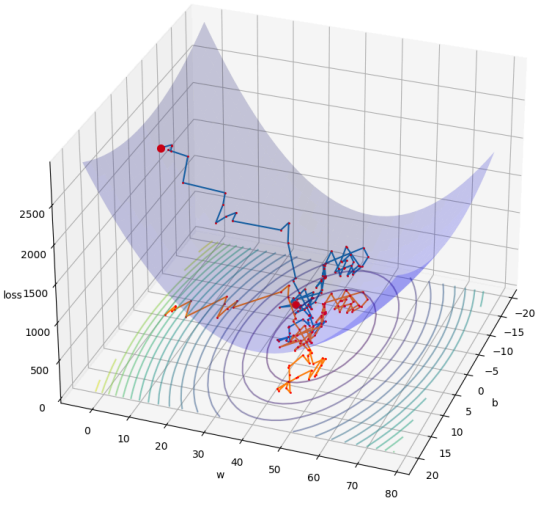


- compute the gradient on the whole dataset for having a better approximation
- but rapidly inefficient for large n (computationally expensive)

Reminder on gradient descent methods

- **Stochastic gradient descent**: approx. the gradient (and update) with every sample

$$\nabla LL(\mathbf{W}) = \nabla_{\mathbf{w}} L(y_i, h_{\mathbf{W}}(\mathbf{x}_i))$$
$$\mathbf{W}_{k+1} \leftarrow \mathbf{W}_k - \eta \nabla LL(\mathbf{W}_k)$$



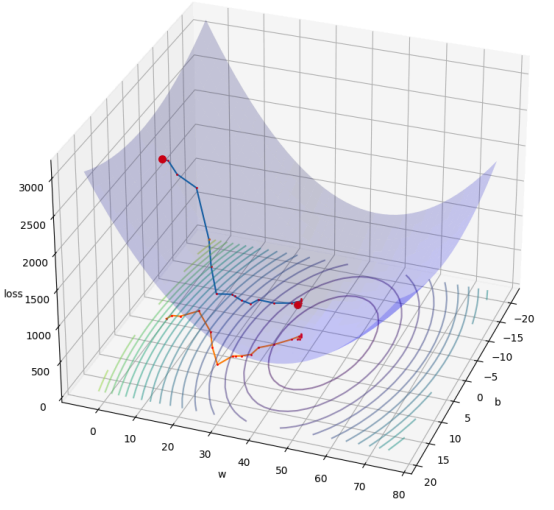
- iterations computationally faster than batch gradient descent
- but can be very noisy and convergence can be very slow

Reminder on gradient descent methods

- **Mini-batch gradient descent**: approx. the gradient on a subset of the dataset

$$\nabla LL(\mathbf{W}) = \frac{1}{m}\sum_{i=1}^{m} \nabla_{\mathbf{w}} L(y_i, h_{\mathbf{W}}(\mathbf{x}_i))$$

$$\mathbf{W}_{k+1} \leftarrow \mathbf{W}_k - \eta \nabla LL(\mathbf{W}_k)$$



- increasing m reduces the noise and speeds up convergence
- but setting m and η can be tricky

Optimizers

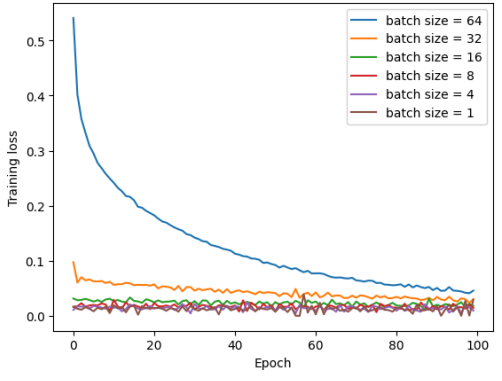
- Modern optimizers improve the GD with various techniques [1]:
  - **Momentum**: add inertia to the gradient to avoid oscillations
  - **Adaptative learning rate**: adjust the learning rate at each iteration
  - **Scheduling (decay)**: reduce the learning rate every few epochs to avoid overshooting
- The three most popular optimizers are **Adagrad**, **RMSprop** and **Adam** [2]
- They help to reach convergence but do not avoid the need to tune hyperparameters

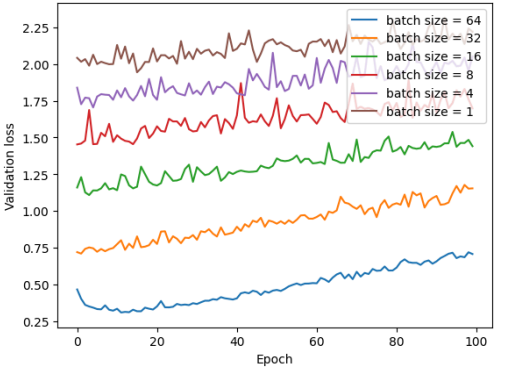
(1) We won't go into detail about all these techniques, as they are generally well implemented in libraries.
(2) Adam is the default choice most of the time

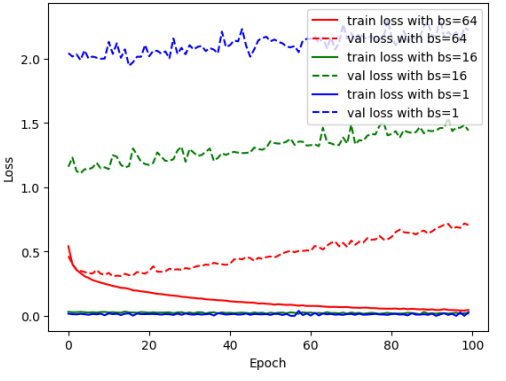
Hyperparameter tuning

In practice, we usually...

- set the batch size to the largest that fits in memory
- set a maximum number of iterations depending on computing resources
- monitor the evolution of the training and validation loss to see what's happening (1)
- adjust architecture and hyperparameters depending on the result







(1) **always plot your losses!**

Hyperparameter tuning

Common symptoms when training a (deep) NN are:

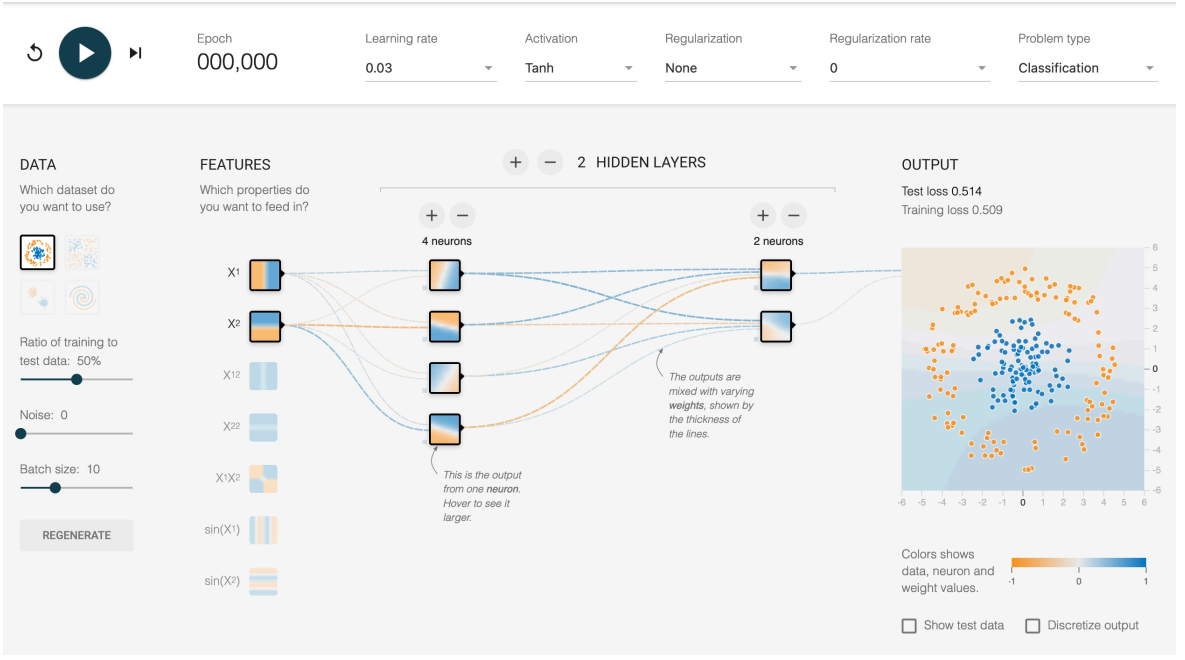
- the training loss is decreasing nicely but the validation loss is not
- the training loss is decreasing so slowly that we don't know if it will converge
- the training loss is oscillating a lot and global trends are not clear (training instability)
- the training loss is not decreasing (the NN is not learning)
- the training loss is increasing (?)

Can you suggest possible cause and potential solution for each of these problems? [1]

(1) Check Claude Fable 5 answers here

Playground

Let's play: https://playground.tensorflow.org/



Takeaways

DNN can model complex patterns [(1)], but they require a lot of data and computing resources to train

The backpropagation algorithm allows the use of gradient descent methods for training

Activation functions play a crucial role in the performance of DNN

Modern optimizers can help improve convergence and stability during training, but **hyperparameter tuning and monitoring of training/validation losses are still essential for successful training of deep neural networks**

(1) cf. "Universal Approximation Theorems" that state that neural networks with a certain structure can, in principle, approximate any continuous function to any desired degree of accuracy